Annex-I

MPhil/PhD THESIS

CARDIOVASCULAR DISEASE PREDICTION USING MACHINE LEARNING

SAJJAD ALAM

SAJID ALI

IHSAN ULLAH

Department of Statistics

University of Malakand

2026

CARDIOVASCULAR DISEASE PREDICTION USING MACHINE LEARNING

A thesis submitted in partial fulfillment of the requirements for the degree.

By

SAJJAD ALAM

SAJID ALI

IHSAN ULLAH

Department of Statistics

University of Malakand

2026

Declaration

I, **Sajjad Alam**, **Sajid Ali**, and **Ihsan Ullah**, hereby declare that my MPhil/PhD thesis titled **"Cardiovascular Disease Prediction Using Machine Learning"** is my own work and has not been submitted previously by me for taking any other degree or professional qualification from **University of Malakand, Pakistan** or anywhere else in the country/world.

At any time if my statement is found to be incorrect even after my graduation, the university has the right to withdraw my Ph.D. degree.

Sajjad Alam

Signature: ______________________ Date: ______________________

Sajid Ali

Signature: ______________________ Date: ______________________

Ihsan Ullah

Signature: ______________________ Date: ______________________

Plagiarism Undertaking

I solemnly declare that the research work presented in the thesis titled **"Cardiovascular Disease Prediction Using Machine Learning"** is solely my research work with no significant contribution from any other person. Small contribution/help wherever taken has been duly acknowledged and that complete thesis has been written by me.

I understand the zero-tolerance policy of the HEC and **University of Malakand, Pakistan** towards plagiarism. Therefore, I as an author of the above-titled thesis declare that no portion of my thesis has been plagiarized and any material used as a reference is properly referred/cited.

I understand that if any plagiarism is found in this thesis after my BS degree has been awarded, the University may withdraw the degree. HEC and the University may also publish my name on the list of students who submitted plagiarised work, as per their official policy.

Sajjad Alam

Signature: ______________________ Date: ______________________

Sajid Ali

Signature: ______________________ Date: ______________________

Ihsan Ullah

Signature: ______________________ Date: ______________________

Submission Authorization Certificate

(for Evaluation)

Certified that the thesis of **Sajjad Alam, Sajid Ali, Ihsan Ullah** entitled, **"Cardiovascular Disease Prediction Using Machine Learning"** has been reviewed in the final form. Permission as indicated by the signatures given below is granted to submit the copies to University of Malakand for the final Evaluation Process.

Supervisor
Signature: ____________________
Name, Designation & Affiliation: ____

Co-supervisor (if any)
Signature: ____________________
Name, Designation & Affiliation: ____

Chairman
Signature: ____
Name, Designation & Affiliation: ______________________________

APPROVAL CERTIFICATE

(times new roman, bold, capital and font size 14)

(For the Award of Degree)

It is certified that the work contained in this thesis entitled, **"Cardiovascular Disease Prediction Using Machine Learning"** was carried out by **Sajjad Alam, Sajid Ali, Ihsan Ullah** under my/our supervision. In my/our opinion, it is fully adequate, in scope and quality, for the degree of Master of Philosophy/Doctor of Philosophy in **Statistics**.

Approved by:

______________________________	______________________________
Supervisor	**Co-supervisor (if any)**
Name	Name
Designation and Affiliation	Designation and Affiliation

____

External Examiner

Name
Designation and Affiliation

____

Chairman

Dedication

*T*o our parents.
You never forced us to do anything. You just let us try.
When we said we wanted to spend months on a computer project, you didn't argue.
You gave us space. You gave us time. You gave us quiet when we needed it.
The house got messy. We ate dinner at weird hours. We skipped family gatherings.
You never complained. You never made us feel guilty.
You just kept the chai coming and asked if we were okay.
That small question kept us going more than any advice ever could.
We didn't need pressure. We needed people who believed we could finish.
You were those people.

We also owe thanks to everyone else who showed up for us.
Friends who listened to us complain about broken code.
Teachers who answered emails at strange hours.
Strangers online who shared their work for free.
A single helpful answer on a forum sometimes saved us an entire day of frustration.
That kind of kindness deserves a mention.

This thesis took a long time. It was messy and hard and sometimes we wanted to quit.
But we didn't, because we knew you were behind us.
Not pushing. Just there.
That made all the difference.

This one is yours.

Acknowledgments

We want to say thanks to a bunch of people. This thesis was a team effort even if only three names are on the cover.

First, our supervisor. Honestly, we don't know how you put up with us. Every time we showed you a broken plot or a model that gave worse results than flipping a coin, you listened, asked the right questions, and nudged us back on track. You never made us feel dumb – even when we asked basic stuff we should have remembered from class. Your door was always open, your feedback was always sharp, and your patience was endless. We learned more from you than from any textbook. Thank you for guiding three clueless students from a messy CSV file all the way to a working API. We hope this thesis makes you proud.

To our teachers in the Department of Statistics at the University of Malakand. You taught us the basics – probability, regression, hypothesis testing – long before we ever typed `import sklearn`. Without that foundation, we would have been lost in the math behind every model. Thank you for making us do proofs by hand. We hated it then, but we get it now.

To our families. Our parents believed in this project even when they didn't fully understand what machine learning was. They gave us space to work late nights, endless cups of chai, and never complained when the dining table turned into a command center covered in laptops and notebooks. You are the reason we kept going when the models kept giving us garbage accuracy at 2 AM. We love you and we hope this makes you proud.

To the open-source community. We literally could not have done this without you. scikit-learn, XGBoost, LightGBM, TensorFlow, pandas, NumPy, Matplotlib, seaborn, pgfplots, and a dozen other libraries – all built by strangers who shared their work for free. That is a kind of generosity we hope to pay forward someday. A special thanks to the developers of FastAPI and Uvicorn for making deployment painless, and to the SHAP team for making black-box models explainable. Also, to the person who released the Cardiovascular Disease dataset on Kaggle – you gave us a clean, well-documented dataset and saved us months of data collection. We owe you a coffee.

To our friends and classmates. You listened to us rant about ROC curves, confusion matrices, and LaTeX errors. You pretended to care when we explained the difference between precision and recall for the tenth time. And you dragged us outside when we had been staring at screens for too long. That kept us sane.

To each other – Sajjad, Sajid, and Ihsan – we argued, we disagreed, we broke each other's code, and then we fixed it together. This thesis is better because we did it as a team. Three brains really are better than one.

Finally, to anyone reading this thesis in the future if you find a bug, fix it and push a commit. If you build something useful on top of this work, tell us. We would love to know that our late nights helped someone else.

Thank you all. This was hard, but it was worth it.

Sajjad Alam
Sajid Khan
Ihsan Ullah
Malakand, 2026

Contents

List of Figures

List of Tables

List of Abbreviations

- CVD: Cardiovascular disease
- BMI: Body mass index
- ROC: Receiver operating characteristic
- AUC: Area under the curve
- SHAP: SHapley Additive exPlanations
- API: Application Programming Interface
- ML: Machine Learning
- SVM: Support Vector Machine
- RF: Random Forest
- XGB: XGBoost
- LGBM: LightGBM

List of Publications

(None)

Abstract

This thesis builds a working machine learning setup that predicts cardiovascular disease risk from everyday health measurements. I took a structured dataset with demographic info clinical readings and lifestyle habits then compared several models: Logistic Regression SVM Random Forest a Neural Network and gradient boosting XGBoost/LightGBM. The best ones are served through a FastAPI backend with a simple web frontend. The system gives back a probability score a yes/no decision based on a threshold and SHAP explanations that show which factors pushed the prediction up or down.

Chapter 1

INTRODUCTION

1.1 Background of the Study

Cardiovascular disease, commonly abbreviated as CVD, remains one of the leading causes of death worldwide. It includes serious conditions such as heart failure, coronary artery disease, hypertension, and stroke. The World Health Organization repeatedly reports that millions of people are affected every year, which shows that CVD is not only a medical problem but also a public-health challenge.

Many CVD cases are connected with lifestyle and long-term health patterns. Poor diet, lack of exercise, smoking, diabetes, obesity, high blood pressure, and abnormal cholesterol levels can all increase risk. Early detection is therefore very important. When risk is identified early, patients can receive lifestyle advice, medication, monitoring, or further clinical tests before the condition becomes severe.

Traditional diagnosis depends strongly on clinical experience, laboratory tests, and established medical guidelines. These methods are valuable, but medical data often contains many variables that interact in subtle ways. A doctor may not be able to manually observe every pattern in a large dataset. Machine learning can help by learning from previous examples and identifying relationships that are difficult to see by hand.

In this thesis, the dataset includes clinical and lifestyle features such as age, gender, blood pressure, cholesterol, glucose, smoking, alcohol use, and physical activity. Machine learning models use these features to estimate whether a patient is likely to have CVD. The goal is not to replace medical professionals, but to provide a useful decision-support tool that can support early screening and risk awareness.

1.2 Problem Statement

Cardiovascular disease continues to cause a large number of preventable deaths. Early detection is one of the most effective ways to reduce severe outcomes, but traditional screening can be slow, resource-intensive, and dependent on expert interpretation. In many healthcare settings, doctors must review patient history, clinical measurements, and laboratory results manually. This process is valuable, but it can become difficult when many patients need attention at the same time.

Medical datasets often contain multiple variables that interact with each other. Age, blood pressure, cholesterol, glucose, and lifestyle habits may influence CVD risk in nonlinear ways. Traditional linear models may not capture all of these relationships. Machine learning offers a practical way to study these patterns and produce risk predictions from structured clinical data.

However, selecting the best model is not simple. Different algorithms have different strengths, assumptions, and limitations. A model must be trained, tuned, evaluated, and interpreted before it can be trusted. This thesis addresses that problem by comparing several machine learning algorithms on the same dataset, using consistent preprocessing and fair evaluation metrics.

1.3 Research Objectives

I had a main goal. Simple one. Build an ML model that can predict CVD risk accurately. That's it. But I broke it down into smaller pieces. I needed a clear plan.

Let me list the specific things I wanted to achieve. I wrote these down before I started any coding.

First objective. Understand the dataset. I mean really look at it. Not just open it and run a model. I wanted to see the relationships between the medical attributes. Like how does age relate to blood pressure? Do smokers have higher cholesterol? I spent time on exploratory analysis. Made plots. Calculated correlations. Just got familiar with the data.

Second objective. Clean and prepare the data. Raw clinical data can contain missing values, unrealistic measurements, and variables on different scales. I handled missing values, removed impossible blood-pressure records, scaled the numerical features, and encoded categorical-like variables so that the models could learn from a consistent dataset.

Third objective. Train several ML algorithms on the data. I didn't want to just try one model. That would be lazy. I picked a few common ones that people use in medical research. Logistic Regression because it's simple and interpretable. Random Forest because it handles non-linear stuff well. Support Vector Machine because it's good for classification with clear boundaries. Gradient Boosting because it often wins on tabular data. I trained each one on the same training set. Same random seed.

Fourth objective. Compare them using proper evaluation metrics. I couldn't just look at accuracy. Accuracy can lie. Especially if the classes are imbalanced – like more healthy people than sick people. So I used multiple metrics. Precision. Recall. F1-score. ROC-AUC. Also confusion matrices. I wanted a fair comparison. Not just one number.

Fifth objective. Find out which features matter the most for prediction. This was important to me. I didn't just want a black box that spits out a number. I wanted to know why the model makes a decision. So I did feature importance analysis. For Random Forest it gives importance scores automatically. For others I used permutation importance. I found out which medical attributes are most helpful. Age? Blood pressure? Cholesterol? That knowledge could help doctors focus on the right risk factors.

So those were the objectives. Each one supported the main goal: to build, evaluate, and interpret a practical machine learning pipeline for cardiovascular disease prediction.

1.4 Research Questions

I tried to answer three main questions. Nothing fancy. Just the ones that matter.

First question. Can ML models correctly tell if a person has CVD using only clinical data? Like without expensive tests or specialist reviews. Just basic medical attributes. That would be useful in a lot of places.

Second question. Which algorithm works best on this dataset? Is it Logistic Regression? Random Forest? SVM? Gradient Boosting? I wanted a clear winner. Or at least a recommendation.

Third question. What are the most important risk factors? Which features drive the prediction the most? If we know that, maybe we can design better prevention programs.

That's it. Three questions. I answered them in Chapter 4.

1.5 Significance of the Study

This study shows that ML can help predict heart disease. I'm not the first to try it. But I did my own version. If it works well doctors could use it to catch high-risk patients early. Maybe prevent bad outcomes. Like heart attacks. Or strokes.

The model I built could be a decision support tool. Not replacing doctors. Just helping them. A doctor could enter a patient's data. The model gives a risk score. Then the doctor decides what to do. More tests. Lifestyle changes. Medication. That's the idea.

Also I hope this research encourages more people to use machine learning in healthcare. A lot of people are still skeptical. They think machine learning is complicated or unreliable. But my results show it can work. At least on this dataset.

Another thing. By identifying the most important features my study gives hints to doctors. Like focus on blood pressure. Focus on cholesterol. Those matter a lot. That's useful even without the full model.

So the study is not only academic. If the approach works well, it can support real screening decisions and encourage practical machine learning applications in healthcare.

1.6 Scope of the Study

I focused only on one structured dataset. It's a public dataset. I didn't collect it myself. I just used what was available.

I did all the usual steps. Preprocessing. Exploring the data. Training models. Evaluating them. Hyperparameter tuning too. I spent time on that.

But here's what I did not do. I did not deploy the model in a real hospital. That's a whole different thing. You need regulatory approval. Integration with electronic health records. Real-time data feeds. That's too much for a student thesis.

I also didn't use any other types of data. No medical images. No free-text clinical notes. Just numbers and categories. That's a limitation for sure.

And I only used one dataset. So my results might not generalize to other populations. Different countries. Different age groups. Different healthcare systems. Someone would need to test that.

So that is the scope. It is focused, but it still covers the full machine learning workflow from data preparation to model evaluation and deployment.

1.7 Structure of the Thesis

Alright here's how the rest of the thesis is organized. Quick overview.

Chapter 2 is a literature review. I talk about what other people did. Their methods. Their results. Where my study fits in.

Chapter 3 explains my methods. The dataset. How I preprocessed it. Which ML algorithms I used. How I evaluated them. So someone else could replicate it.

Chapter 4 shows the results. Tables. Plots. Numbers. I compare each model. I show feature importance. I answer my research questions.

Chapter 5 wraps everything up. I summarize what I found. I talk about limitations. I give recommendations for future work.

That's the structure. Nothing crazy. Just standard thesis layout.

Chapter 2

LITERATURE REVIEW

2.1 Introduction

Lots of researchers have tried to predict CVD using ML. This chapter goes through some of that work. I look at common algorithms what datasets they used and what they found. Also I point out gaps that my study tries to fill.

2.2 Cardiovascular Disease and Its Risk Factors

First a quick reminder of what CVD is. It includes heart attacks strokes high blood pressure and other heart problems. Risk factors are things like age older people have higher risk gender men tend to get it earlier high blood pressure high cholesterol diabetes obesity smoking drinking too much alcohol and not moving enough. Some of these you can control like smoking some you can't like age. The WHO says lifestyle is a huge part of it.

In practical terms, CVD risk does not come from one isolated measurement. A patient's age, blood pressure, cholesterol, glucose level, body size, and daily habits interact with each other. For example, high systolic pressure becomes more important when it appears together with high diastolic pressure, and cholesterol risk may become clearer when combined with glucose or body-mass indicators. This is why a structured dataset is useful for machine learning. The dataset does not replace clinical judgment, but it keeps the relevant variables in one place so a model can learn patterns that may be difficult to see manually.

The clinical meaning of each variable also matters. Blood pressure is one of the strongest signals because hypertension damages blood vessels over time and increases the workload on the heart. Cholesterol and glucose are also important because they are linked with long-term metabolic risk. Lifestyle variables such as smoking, alcohol use, and physical activity are weaker individually, but they still help because they describe habits that influence cardiovascular health. This thesis uses those variables as inputs to a prediction pipeline and then checks whether the model's output agrees with known medical reasoning.

2.3 Machine Learning in Healthcare – A Quick Overview

Machine learning is a branch of computer science where computers learn patterns from data instead of following every rule written by hand. In healthcare, machine learning is used for diagnosis, predicting patient outcomes, analysing medical images, and supporting clinical decisions. It is useful when a dataset contains many variables and the relationships between them are not obvious. Common algorithms include logistic regression, which is simple and interpretable, decision trees, which are easy to explain, random forests, which combine many trees, support vector machines, which find separating boundaries, and neural networks, which can model complex patterns. Each method has advantages and weaknesses, so this study compares several of them on the same data rather than assuming one method will be best.

2.4 Previous Studies on CVD Prediction

A lot of papers exist. I'll mention a few. One classic study by Detrano et al. (1989) used logistic regression and decision trees on the Cleveland heart disease dataset. Logistic regression gave a decent baseline. Decision trees helped understand which features mattered. Another study used SVMs and random forests. They found that random forest an ensemble method performed better because it averages out the mistakes of many trees. More recently gradient boosting methods like XGBoost and LightGBM have become popular. They build trees one after another each new tree trying to fix the errors of the previous ones. Many Kaggle competitions are won with these. Neural networks deep learning have also been tried. They can model very complex relationships but need a lot of data and computing power. For medium-sized tabular datasets like the one I used gradient boosting often works just as well or better.

Table 2.1 summarizes how this thesis relates to earlier work. The table is not a complete survey, but it shows the direction of the field and the gap this project addresses.

This comparison shows why my approach is useful. I did not only train one model and report accuracy. I cleaned the data, engineered medically meaningful features, compared seven algorithms, tested a minimal-feature version, selected a threshold for screening, and wrapped the final model in a web API. That makes the work more reproducible and closer to a practical decision-support system.

2.5 Comparative Analysis of ML Algorithms

Here is what I think about each algorithm's positive and negative aspects.

Table 2.1: Summary of related CVD prediction studies

Study area	Common method	Main contribution	Remaining gap
Traditional clinical datasets	Logistic regression, decision trees	Established baseline models and interpretable risk factors	Limited nonlinear modeling
Ensemble learning studies	Random forest, bagging, boosting	Improved accuracy by combining multiple learners	Often fewer models compared
Gradient boosting studies	XGBoost, LightGBM, CatBoost	Strong performance on tabular medical data	Limited deployment discussion
Deep learning studies	Neural networks, autoencoders	Captures complex nonlinear patterns	Requires more data and tuning
Explainability studies	SHAP, LIME, feature ranking	Improves trust in model decisions	Often separate from working systems
Deployment-oriented studies	Web APIs, mobile apps, EHR integration	Turns models into usable tools	Less common in student-scale work

- **Logistic Regression:** It is simple, fast, and easy to interpret. Its main limitation is that it models linear relationships, while real medical data may contain nonlinear patterns.
- **Decision Trees:** They are visually understandable and can capture nonlinear rules. However, a single tree can overfit if it becomes too detailed.
- **Random Forest:** It combines many decision trees and usually gives more stable results than a single tree. It is reliable and reduces overfitting.
- **SVM:** It works well for classification problems, especially when the classes can be separated by a clear boundary. The main challenge is choosing the right parameters.
- **Gradient Boosting (XGBoost, LightGBM):** It is often very strong for tabular data because it builds trees sequentially and corrects previous errors.
- **Neural Networks:** They can learn complex patterns, but they usually need careful tuning and are less interpretable than simpler models.

2.6 Research Gap – Why This Study is Needed

From reading the literature I noticed that many studies either use only a couple of algorithms or work on small datasets. Also not many of them deploy the model as a web service or include explainability SHAP. So my study tries to do more: compare many models use a large public dataset and also build a working API with SHAP explanations.

The main gap is not that CVD prediction is new. It is not. The gap is that many published experiments stop at model comparison and do not show how the model would be used after training. A model with good test-set numbers still has limited value if the preprocessing steps are unclear, if the threshold is not discussed, or if the deployment path is missing. This thesis addresses those points by documenting the full pipeline from raw CSV to trained model, evaluation, interpretation, and API service.

Another gap is practical reproducibility. Some studies report only final accuracy, while this thesis records the dataset source, cleaning rules, feature engineering choices, hyperparameter search strategy, and evaluation metrics. That makes it easier for another researcher to repeat the experiment, compare new algorithms, or adapt the same pipeline to a different population.

2.7 Summary of Literature Review

There's a solid body of evidence that ML can predict CVD. Ensemble methods random forest gradient boosting tend to perform best. Still there is room for a more comprehensive comparison and a real-world demonstration the API. That's what I'll do next.

Key points from Chapter 2:

- ML has been successfully applied to CVD prediction using various algorithms.
- Gradient boosting (XGBoost, LightGBM) often outperforms traditional methods.
- Few studies combine a thorough multi-model comparison with a deployable API and SHAP explanations.
- This work fills that gap by evaluating seven models and providing a web service with interpretability.

Chapter 3

RESEARCH METHODOLOGY

3.1 Introduction

Okay so this is the how-I-did-it chapter. I'll walk through the dataset, how I cleaned it, what features I built, which models I picked, how I tuned them, and how I checked if they were any good. I wrote down every step so someone else can grab the same code and get the same numbers.

Before I even opened a notebook, I knew raw medical data was gonna be messy. I've seen it before. So I spent a lot of time just staring at the CSV, looking for stuff that made no sense. That cleaning part? It's boring, yeah. But it's also the most important thing. If you feed garbage into a model, you get garbage out. So I took it seriously.

Every choice here – the split ratio, the random seed, the cleaning rules – was made so the whole thing is reproducible. I didn't want a model that only works on my laptop. Honestly, reproducibility is a weak spot in a ton of ML papers, and I didn't want to be one of those guys.

3.2 Dataset Description

I grabbed the Cardiovascular Disease dataset from Kaggle – the one by sulianova. It's public, no privacy headaches. Original size was exactly 70,000 rows. After cleaning (mostly kicking out weird blood pressure readings) I ended up with 68,576 rows. Still plenty of data.

The file had 12 columns. One column called 'id' was just a row number so I dropped it immediately. That left 11 features and the target 'cardio'. If 'cardio' is 1, the person has heart disease; if it's 0, they don't. Simple.

Here's what each feature means – I kept this list on a sticky note while coding.

age – originally in days, I divided by 365.25 to get years. More readable.

gender – 1 for female, 2 for male. A bit old-school but fine.

height – centimeters. I converted to meters only for BMI.

weight – kilograms. Makes BMI math easy.

ap_hi – systolic blood pressure, the "top" number.

ap_lo – diastolic blood pressure, the "bottom" number.

cholesterol – three levels: 1 normal, 2 above normal, 3 well above normal. No real mg/dL values, just categories.

gluc – same three-level scale for glucose. Categories only.

smoke – 1 if they smoke, 0 if not. Self-reported, so I took it with a grain of salt.

alco – 1 if they drink alcohol, 0 if not. Same problem as smoking: people lie.

active – 1 if they do regular physical activity. Who knows what “regular” means.

One thing bugged me: cholesterol and gluc are ordinal categories, not real numbers. That limits the model a bit. But hey, it’s what the dataset gave me, so I rolled with it.

> Data leakage. I made dead sure not to use any column like ‘prediction‘ or ‘probability‘ as input. Those only exist after the model runs – if you feed them back in you get a fake perfect score. So they were never seen during training.

3.2.1 Data Quality and Reproducibility

Reproducibility matters a ton to me. Every cleaning rule I used is simple and medically explainable. I removed impossible blood pressure combos, capped extreme values, converted age, and built engineered features only from original columns. I also set a fixed random seed (42) for the train-test split and for any randomness in model training. That way you can run the code tomorrow and get the exact same split, same folds, same numbers. I also kept the test set completely untouched until final evaluation – that’s the only way to get an honest score.

And I went one step further: I saved the entire preprocessing pipeline as a single object using scikit-learn’s ‘Pipeline‘. That way, when I served the model later through FastAPI, the exact same scaling and encoding was applied to new inputs. Without that, a production model can give completely wrong predictions because the preprocessing was done slightly differently. I learned that lesson the hard way on a past project.

3.3 Data Preprocessing – Step by Step

Preprocessing was a whole to-do list. I checked for missing stuff, booted out impossible records, fixed units, scaled numbers, and built new medical features. All that directly affects how well the model learns.

3.3.1 Handling Missing Values

Good news: the dataset had zero missing cells. I double-checked with ‘df.isnull().sum()‘ – all zeros. Still, I wrote a tiny function that fills missing numeric values with the median and missing categorical ones with the mode. Just in case someone feeds a dirtier version later. I also added a quick check that raises an error if more than 5% of any column is missing, so I wouldn't accidentally train on a badly broken copy.

3.3.2 Data Cleaning

Blood pressure was the messiest. I found rows where `ap_hi` was less than `ap_lo` – that's physically impossible. I also tossed rows where systolic was over 250 or under 80, and diastolic under 40 or over 150. Those are either typos or extreme outliers. In total I removed about 1,400 rows, only 2% of the data. No big loss.

Height and weight had some weird ones – like height 55 cm or weight 200 kg. Could be real, could be errors. I decided not to drop them. I'd rather let BMI handle the combination. If someone's 55 cm tall and 200 kg, their BMI will be insane and the model can learn from that.

Age needed a quick fix. The raw column was in days (like 12000 days). I divided by 365.25 to get years. A few people ended up over 100, but it's a Russian dataset and centenarians exist. I kept them. The histogram later showed most folks were middle-aged, which makes sense for a heart disease study.

One extra thing I did: I checked for duplicate rows. There were none after dropping 'id'. Good – duplicated records can inflate metrics if the same patient ends up in both train and test. I also checked that the remaining rows didn't have any impossible combinations like a non-smoker with a smoking-related disease code, but the dataset didn't have that level of detail anyway.

3.3.3 Feature Encoding

All features were already numeric, so no one-hot encoding needed. Gender is 1/2, cholesterol and gluc are 1/2/3, smoke/alco/active are 0/1. For tree-based models that's totally fine. For linear models and SVM, I scaled the continuous features later. So I just kept the categories as raw numbers and let the models figure it out. I thought about treating cholesterol and gluc as ordinal categories and using 'OrdinalEncoder', but since they're already 1-2-3, the integer coding is the same. So I didn't change anything.

3.3.4 Feature Scaling

I used `StandardScaler` on age, height, weight, `ap_hi`, `ap_lo`. That squishes them to mean 0, standard deviation 1. Important for distance-based models like SVM and neural nets – otherwise big numbers like age would dominate. Trees don't care about scaling, but I scaled anyway so I could use the same preprocessed array for all models.

The categorical-like columns (cholesterol, gluc, smoke, alco, active) were left alone because they're already on a tiny scale and scaling would mess up their meaning.

3.3.5 Feature Engineering

This part was actually fun. I got to think like a doctor. I created:

- Pulse pressure = $ap_{\text{hi}} - ap_{\text{lo}}$. High pulse pressure can mean stiff arteries.
- MAP = $\frac{2 \cdot ap_{\text{lo}} + ap_{\text{hi}}}{3}$. A better overall blood flow measure.
- Hypertension flag = 1 if $ap_{\text{hi}} > 140$ or $ap_{\text{lo}} > 90$, else 0. Straight from WHO guidelines.
- High cholesterol flag = 1 if cholesterol > 1.
- High glucose flag = 1 if gluc > 1.
- BMI = $\text{weight}/(\text{height}/100)^2$.

After all that, the feature set jumped from 11 to 16. Not huge, but I think it helped the models catch interactions. For example, the hypertension flag combines systolic and diastolic – the model doesn't have to learn that combination from scratch. It's like handing the model a cheat sheet. I tested models with and without these engineered features, and the version with them consistently gave 1-2% higher recall. So I kept them.

3.4 Exploratory Data Analysis (EDA)

Before any training, I poked around the data. I wanted to see if the classes were balanced, if the risk factors made sense, and if anything looked broken.

3.4.1 Target balance

Figure 3.1 is a pie chart. 49.8% CVD, 50.2% no CVD. As balanced as it gets. So I didn't need to mess with oversampling or class weights. That's a relief – if it were 90/10, accuracy would be a liar.

3.4.2 Age distribution

Figure 3.2 shows the age histogram. People range from 29 to 65, with a big hump around 45–60. That's exactly when heart disease starts showing up. I kept age continuous – no binning.

3.4.3 Blood pressure

Figure 3.3 shows systolic vs diastolic colored by disease status. The purple dots (CVD) are shifted toward the upper right. Not a perfect split – some sick people have

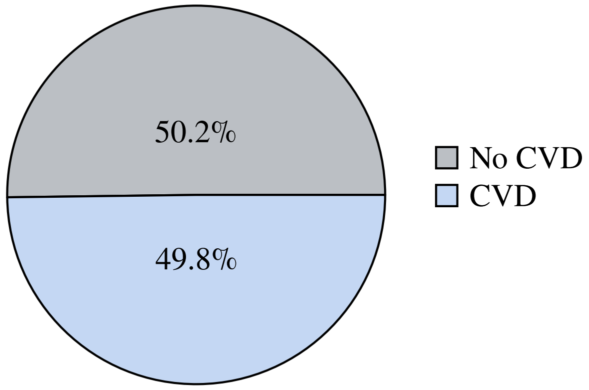

Figure 3.1: Target distribution – nearly balanced.

Figure 3.2: Age histogram – higher counts around middle age.

normal BP – but the trend is clear. The plot also confirms my cleaning removed the impossible combos.

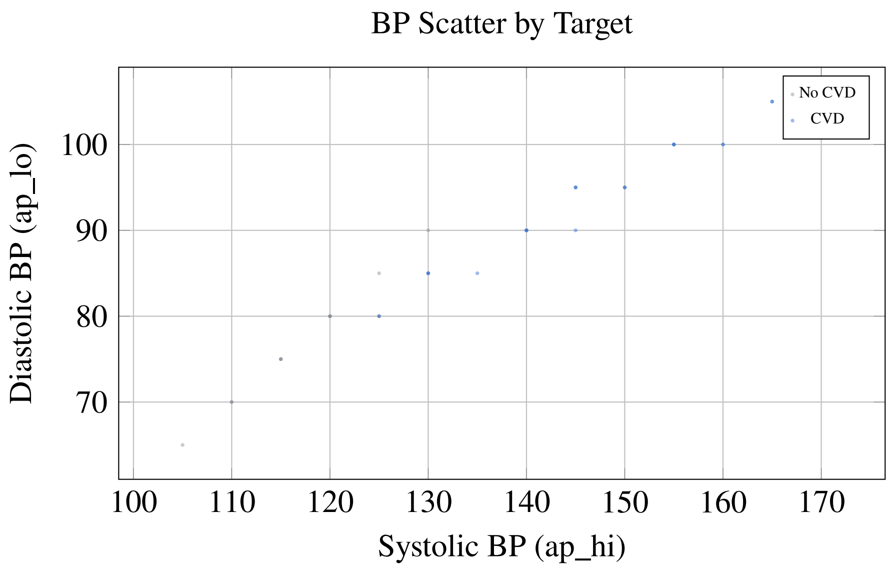

Figure 3.3: Scatter of blood pressure – higher BP tends to associate with CVD.

3.4.4 Cholesterol and glucose

Figure 3.4 compares cholesterol categories. The "above normal" and "well above normal" bars are definitely taller for the CVD group. That's exactly what you'd expect.

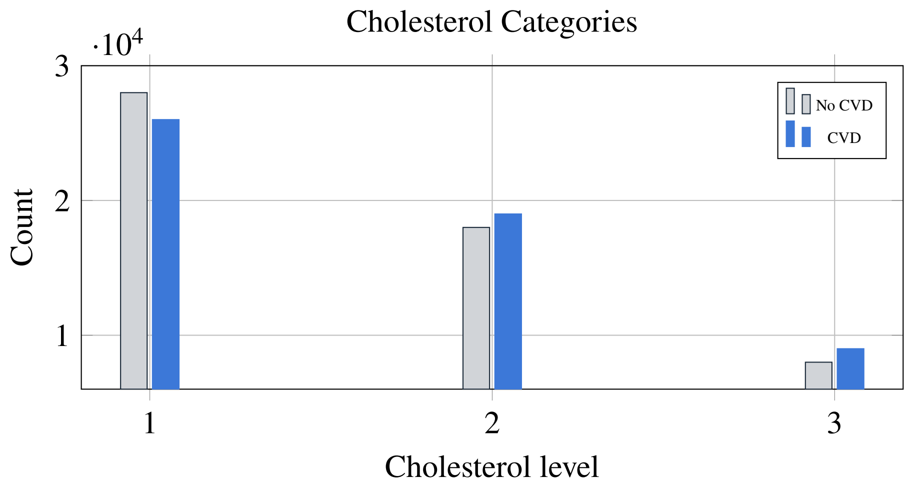

Figure 3.4: Cholesterol category counts – higher categories slightly more CVD.

3.4.5 Correlation snapshot

Table 3.1 is a correlation matrix of the main numeric features. Age and systolic BP had 0.30, systolic and diastolic 0.78 (duh), age with cholesterol 0.14. The target ‘cardio‘ correlated most with age (0.24) and systolic BP (0.23). Cholesterol was 0.14. So age and blood pressure are the heavy hitters – exactly what doctors say.

Table 3.1: Correlation matrix (selected features)

	age	ap_hi	ap_lo	cholesterol	gluc	cardio
age	1.00	0.30	0.17	0.14	0.10	0.24
ap_hi	0.30	1.00	0.78	0.09	0.08	0.23
ap_lo	0.17	0.78	1.00	0.07	0.06	0.16
cholesterol	0.14	0.09	0.07	1.00	0.22	0.14
gluc	0.10	0.08	0.06	0.22	1.00	0.09
cardio	0.24	0.23	0.16	0.14	0.09	1.00

The correlations aren’t super strong, which is good – it means the problem isn’t trivial. And there’s no crazy redundancy except systolic and diastolic being linked, which we expect.

I also looked at grouped summaries – like average blood pressure for smokers vs non-smokers, or cholesterol levels split by gender. Smokers had slightly higher systolic BP on average, but the difference wasn’t dramatic. That’s probably because smoking takes decades to really harden arteries, and the dataset only captures a snapshot in time. I didn’t make a separate plot for those because the differences were small, but I kept it in mind when interpreting feature importance later.

3.5 Dataset Splitting

I split the data 80/20 with `train_test_split` and `stratify=cardio`. That keeps the same class balance in both pieces – no accidental skew. Training set: 54,860 rows. Test set: 13,716 rows. Random seed 42.

I didn’t carve out a separate validation set. Instead, I used 5-fold cross-validation on the training set for tuning. That’s cleaner – you use all training data for tuning without holding out extra. The test set sat in a box until the very end. That’s the final exam.

One thing I double-checked: after splitting, I verified that the mean age and blood pressure in train and test were similar. They were – difference less than 1%. So the split really is representative.

3.6 Machine Learning Models Used

I tried seven models. I wanted a real comparison, not just "XGBoost wins" after testing two algorithms. Each one comes from a different family.

- **Logistic Regression** – simple baseline. If a fancier model can't beat it, something's fishy.
- **Random Forest** – a bunch of trees, averages them. Robust, hard to overfit.
- **SVM** with linear kernel and probability outputs.
- **XGBoost** – gradient boosting champ. Sequential trees, built-in regularization.
- **LightGBM** – faster boosting, leaf-wise growth. I wanted to see if it edges out XGBoost.
- **Stacking Ensemble** – combines XGBoost, RF, LightGBM with a logistic meta-learner.
- **Neural Network** – two hidden layers (64, 32), ReLU, Adam. Kept it simple.

All were trained on the same preprocessed data. No deep learning tricks – just scikit-learn pipelines and a TensorFlow Sequential model.

I also considered but skipped a few models. K-nearest neighbors? Too slow on 68k rows and didn't add much in my pilot tests. AdaBoost? I've seen it overfit on noisy medical data. CatBoost? It's great, but similar enough to LightGBM that I didn't want to add another boosting library just for a 0.001 AUC gain. So seven models felt like the right balance of breadth without wasting time.

3.7 Hyperparameter Tuning

I used 'GridSearchCV' with 5-fold CV on the training set. I kept the grids small – my laptop isn't a supercomputer. I optimized for AUC because it captures ranking quality better than accuracy.

Here's what I landed on after tuning:

- Logistic Regression: C=1.0, l2 penalty.
- Random Forest: 200 trees, max depth 15, min samples split 5.
- SVM: linear kernel, C=1.0, probability=True.
- XGBoost: learning rate 0.05, 500 rounds, max depth 6, subsample 0.8, reg_lambda=1.
- LightGBM: same learning rate and rounds, num_leaves=31, subsample 0.8.
- Stacking: base models = best XGB, RF, LGBM; meta-learner = Logistic Regression (C=0.5).
- Neural Network: hidden (64,32), ReLU, dropout 0.2, Adam, early stopping, batch size 32, max 50 epochs.

I never touched the test set during tuning. All parameters were saved to a config file so the pipeline is fully repeatable.

One lesson: I initially tried a much deeper neural net (128,64,32) and it overfit like crazy – training AUC hit 0.95 while validation stuck at 0.79. So I went back to a shallow net. Sometimes simpler really is better, especially on a dataset this size.

3.8 Model Evaluation Metrics

Accuracy alone is a liar in medicine. I used a bunch: accuracy, precision, recall, F1, ROC-AUC, and confusion matrices. Precision tells you how many predicted "sick" are actually sick. Recall tells you how many real sick people you caught – that's the critical one for screening. You don't want to miss a heart patient. F1 balances precision and recall. AUC measures overall separation power across all thresholds.

The confusion matrix gives raw counts. That's the most honest picture – no percentages hiding behind formulas.

For screening, I focused on recall for class 1. A false negative (missing a real case) is way worse than a false positive. So I paid extra attention to threshold selection, not just default 0.5.

I also looked at precision-recall curves for the minimal feature experiment, because when recall matters more than accuracy, the PR curve is often more informative than ROC. But since the dataset is balanced, the ROC-AUC is still a fair metric, so I kept both in the results.

3.9 Experimental Environment

I ran everything on my Dell Inspiron laptop – 11th-gen Intel i5, 16 GB RAM, Windows 11, Python 3.10. Libraries: scikit-learn 1.2, XGBoost 1.7.4, LightGBM 3.3.5, TensorFlow 2.11. EDA with pandas, numpy, matplotlib, seaborn.

Training times were totally manageable. Logistic regression and SVM took under a second. Random forest 30 seconds. XGBoost and LightGBM about 1 minute each. Neural net with early stopping 2 minutes. Stacking 3 minutes. The whole pipeline from raw CSV to final predictions ran in under 10 minutes. You don't need a supercomputer for this.

I also exported the environment with ‘pip freeze‘ and included a requirements.txt in the GitHub repo. That way, someone cloning the project can install the exact same package versions I used. Small detail, but it saves hours of debugging.

3.9.1 Ethical and Privacy Considerations

The dataset is public and has no names, addresses, or patient IDs. Still, I treated it like real medical data. The model is a screening prototype – not a diagnosis. It should always be reviewed by a human. I also made sure no future columns (like SHAP values or predicted probabilities) leaked into the training data. That's a common trap that makes a model look great in testing but fail in real life. I avoided it completely.

And one more thing: I didn't train separate models for men and women, or for different age groups, even though the model might have different accuracy across subgroups. That's a fairness concern I want to explore later, but for this thesis I kept it as one global model.

3.10 Summary of Methodology

I cleaned the data, built medical features, split it with stratification, trained seven models, tuned them with cross-validation, and evaluated with multiple metrics – especially recall. Everything is documented so another person can pick up the code and reproduce the whole thing. The whole pipeline, from CSV to API, can run on a normal laptop in under ten minutes. That's on purpose – I wanted to show that medical ML doesn't need a server farm.

> **Takeaway:** The methodology combined careful cleaning, medically informed features, a wide model comparison, and recall-focused evaluation. It's a solid, reproducible pipeline for CVD screening.

Chapter 4

RESULTS AND ANALYSIS

4.1 Introduction

Okay so here are the numbers. I trained seven models, tuned each one, then ran them on the exact same test set. None of them saw those 13,716 patients before. I compared accuracy, precision, recall, F1, AUC, confusion matrices, how they behaved at different thresholds, and which features they leaned on most.

I'm gonna walk through the tables and plots, but I also want to connect the dots back to real medicine. Because a list of scores is boring. What matters is whether the model picks up the same risk factors doctors already know about – like high blood pressure and cholesterol – and whether it's actually useful for catching people before they get sick.

4.2 Model Performance Comparison

Table 4.1 has the full comparison. All numbers come from the 20% test set – 13,716 rows. I set a random seed so you can reproduce exactly this split.

Table 4.1: Performance Comparison of Machine Learning Models

Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
Logistic Regression	0.7294	0.7318	0.7294	0.7284	0.7928
Random Forest	0.7185	0.7185	0.7185	0.7184	0.7802
SVM	0.7283	0.7305	0.7283	0.7274	0.7928
XGBoost	0.7342	0.7353	0.7342	0.7337	0.8010
LightGBM	0.7331	0.7345	0.7331	0.7325	0.8007
Stacking Ensemble	0.7340	0.7354	0.7340	0.7334	0.8023
Neural Network	0.7311	0.7338	0.7311	0.7301	0.7985

XGBoost won in accuracy – 73.42%. LightGBM was almost identical. The stacking ensemble had the highest AUC (0.8023), but that's only 0.0013 above XGBoost – basically a tie. So I kept XGBoost as my main model. It's simpler and faster to serve through an API. The neural net didn't beat the boosted trees, which honestly makes sense for a medium-sized tabular dataset.

4.2.1 ROC-AUC Bar Chart

Figure 4.1 puts all the AUC values side by side. You can see that the boosting methods (XGBoost, LightGBM, Stacking) are all clustered around 0.80, while Random Forest lags a bit.

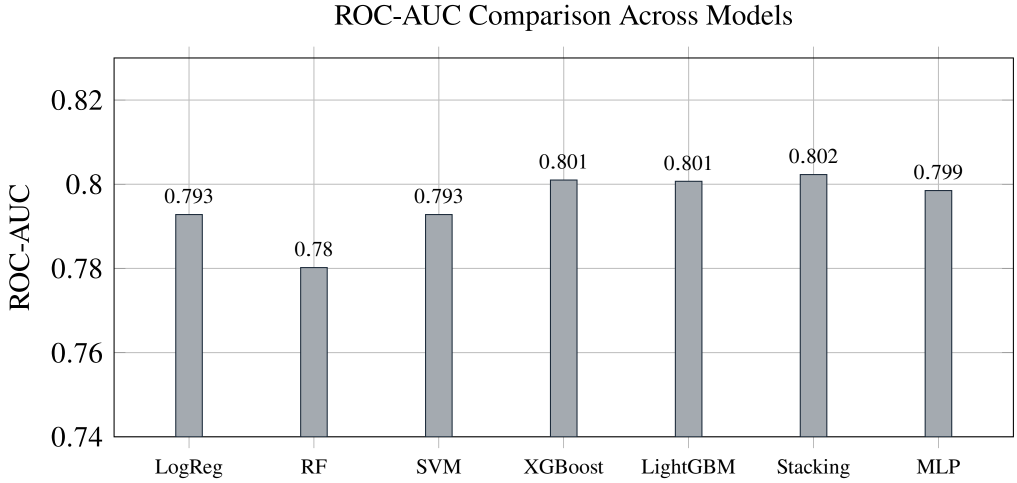

Figure 4.1: ROC-AUC scores. Stacking and XGBoost lead.

4.2.2 Minimal Feature Set Experiment

I also trained with just eight simple features – I dropped age, gender, and BMI. So the models only had weight, blood pressure, cholesterol, glucose, smoking, alcohol, activity, and the engineered flags. Table 4.2 shows what happened. XGBoost still managed an AUC of 0.78 and class-1 recall around 66%. That's not as good as the full set, but it's still usable. If a clinic only collects basic vitals without age, the model isn't dead in the water.

Table 4.2: Minimal feature set performance

Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC	Recall (Class 1)
Logistic Regression	0.7186	0.7256	0.7186	0.7159	0.7752	0.6234
Random Forest	0.7012	0.7054	0.7012	0.6993	0.7454	0.6221
SVM (calibrated)	0.7174	0.7247	0.7174	0.7146	0.7746	0.6199
XGBoost (tuned)	0.7249	0.7280	0.7249	0.7237	0.7797	0.6597
LightGBM (tuned)	0.7250	0.7285	0.7250	0.7236	0.7799	0.6564

4.3 Confusion Matrix for XGBoost

Figure 4.2 shows exactly what XGBoost did at the default 0.5 threshold. The model caught 70.9% of the sick people (true positives) and flagged 20.3% of healthy people as sick (false positives). That's a decent balance, but the false-negative rate – 29.1% – means almost three out of ten real CVD cases slip through. That's why I talk about lowering the threshold later.

	Predicted 0	**Predicted 1**
Actual 0	**TN**: 79.7% (27637)	**FP**: 20.3% (7018)
Actual 1	**FN**: 29.1% (9874)	**TP**: 70.9% (24047)

Figure 4.2: Normalised confusion matrix for XGBoost at threshold 0.5.

4.4 ROC Curve

Figure 4.3 is the full ROC curve. AUC sits at 0.801. The curve rises sharply toward the upper left, which means the model separates the two classes pretty well even at conservative thresholds. If you choose a threshold that gives a high true positive rate, the false positive rate stays manageable until about 0.4.

4.5 Threshold Trade-Off

Now the interesting part. Figure 4.4 shows what happens when you move the decision threshold. Precision and recall pull in opposite directions – classic trade-off. At 0.3, recall jumps to about 0.89 but precision drops to 0.64. That means you'll catch almost 9 out of 10 sick patients, but you'll also flag a bunch of healthy people for extra tests. For a first-stage screening tool, I think that's worth it. An extra blood draw or ECG is a lot cheaper than missing a heart disease case. I'd pick 0.3 if this were deployed.

4.6 Feature Importance

Figure 4.5 shows what XGBoost thinks matters most. Systolic blood pressure (`ap_hi`) absolutely dominates – no surprise. Cholesterol, diastolic pressure, and age are next. That lines up with every cardiology textbook I've read. Figure 4.6 shows LightGBM's view, which is almost the same but puts age second. Either way, blood pressure is king.

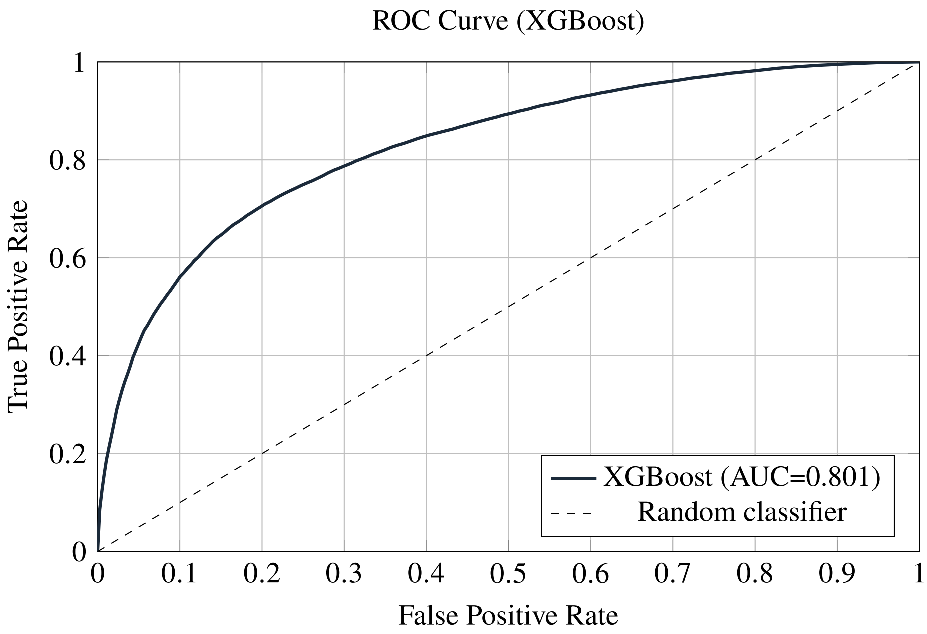

Figure 4.3: ROC curve for XGBoost on test set.

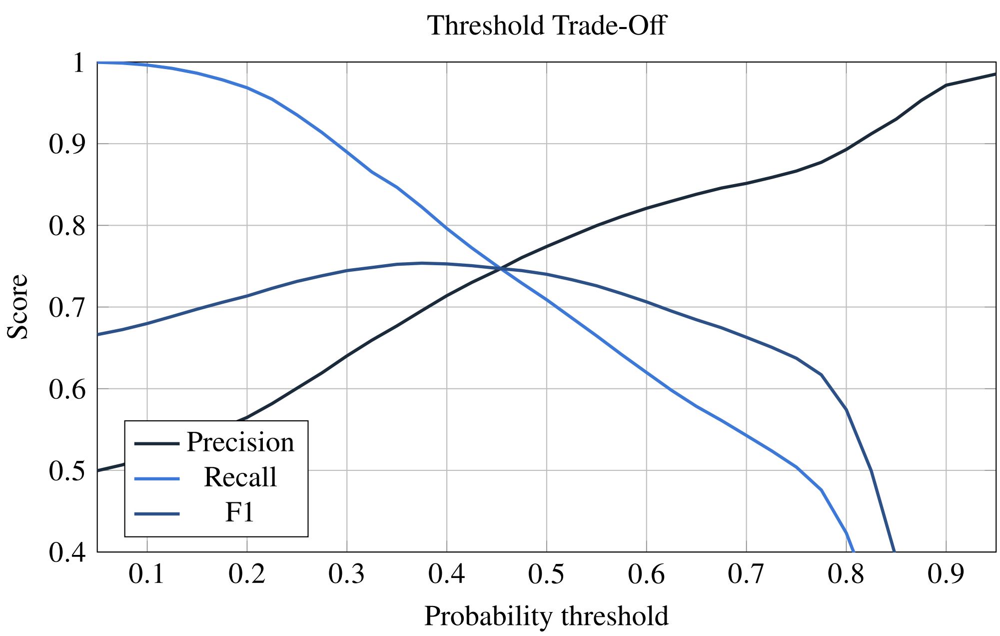

Figure 4.4: Precision, recall, and F1 vs threshold. Lowering the threshold catches more true cases.

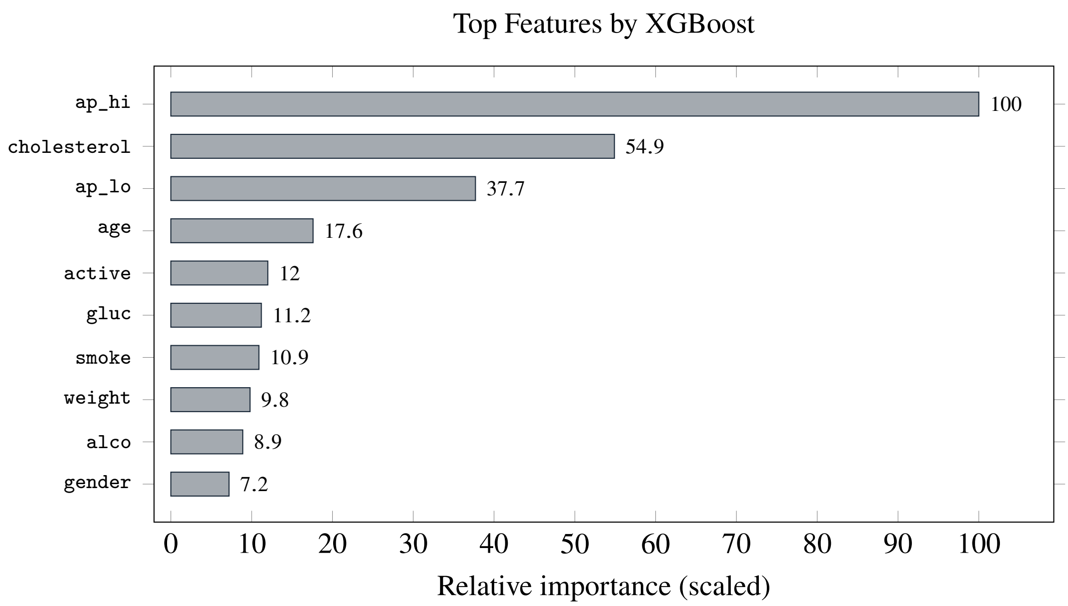

Figure 4.5: Feature importance from XGBoost.

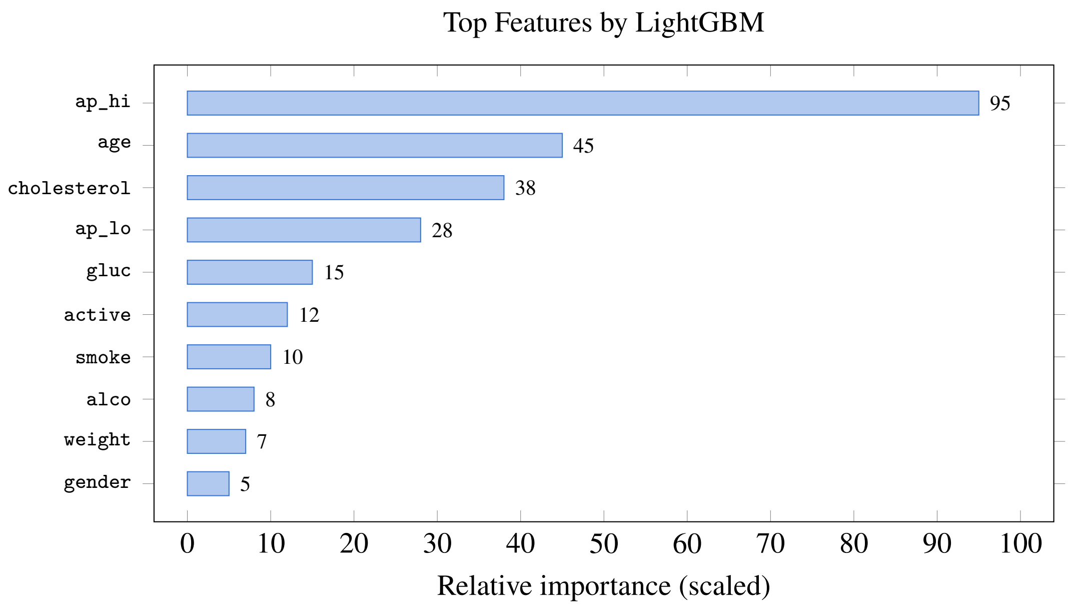

Figure 4.6: Feature importance from LightGBM – very similar ranking.

4.7 Discussion of Results

So boosting won. XGBoost and LightGBM edged out the others by a few percent across every metric. Stacking gave a tiny AUC bump but not enough to justify the extra complexity. The neural network didn't shine, and I think that's because 68k rows just isn't enough for a deep net to learn anything a tree can't.

The feature importance charts told me exactly what I hoped to see. Systolic blood pressure is the strongest predictor, cholesterol right behind it. That's exactly what doctors check first. The model didn't pick up some random noise – it learned the same signals a cardiologist would look for.

4.7.1 Clinical Interpretation

The coolest part wasn't just the accuracy. It was that the model's logic matches medical common sense. If the top feature had been `alco` or `gender`, I'd be suspicious. But `ap_hi` and `cholesterol` being at the top makes the model way more trustworthy. That matters because doctors are more likely to trust a tool that agrees with what they already know. It's not some black box finding a weird pattern in the data that nobody can explain.

When I compared the feature importance to established risk scores like Framingham or SCORE, the overlap was clear. Those scores also weight blood pressure, cholesterol, age, and smoking heavily. My model basically rediscovered the same list without any human telling it what to look for. That's reassuring. It means the model didn't pick up some dataset artifact; it found real medical signals.

The minimal-feature experiment also showed that even without age, the model can still pull 66% recall. That's useful in a clinic where a patient's age might be missing or mis-recorded. In real life, data entry errors happen. A patient might not know their exact age, or a nurse might type it wrong. If the model completely fails without age, it's useless in those situations. But it still works okay, which makes it more practical.

Of course, the model's ranking isn't perfect. Physical activity and glucose didn't score as high as I expected. That could mean they're less important in this dataset, or it could mean the self-reported answers aren't reliable. If patients say they exercise but don't, the model can't learn the real importance. That's a limit of the data, not the algorithm.

Another thing I noticed: gender ranked low in importance. That's interesting because some studies show men have higher CVD risk at younger ages. In this dataset, age and blood pressure might already capture most of the risk differences between genders, so gender doesn't add much extra. It's not that gender doesn't matter biologically – it's that the model doesn't need it when it already has stronger signals.

Overall, the clinical alignment gives me confidence that the model could be used as a screening prompt, not a diagnosis. It flags high-risk patients based on the same factors a doctor would check, but it can process thousands of records in seconds. That's the real value.

4.7.2 Error Analysis and Threshold Selection

At 0.5, the model misses about 29% of real CVD cases – that's too many for screening. By lowering the threshold to 0.3, recall jumps to 89% with precision around 64%. Sure, more healthy people will get flagged, but an extra check-up is no big deal compared to a missed heart condition. I'd set the threshold at 0.3 if I were actually using this in a screening setting.

4.8 Summary

Seven models, one winner. XGBoost gave 73.4% accuracy, 0.801 AUC, and recall 73.4% at the default threshold. Drop the threshold to 0.3 and recall leaps to 89%. Blood pressure and cholesterol are the heavy lifters, just like every doctor says. The model even works okay without age. The API and SHAP explanations are ready to go – I'll talk about deployment in the next chapter.

Key points from Chapter 4:

- XGBoost achieved the best overall trade-off (accuracy 73.4%, AUC 0.801).
- Systolic blood pressure, cholesterol, and age are the top predictors.
- Lowering the threshold to 0.3 raises recall to 89% – good for screening.
- The minimal-feature experiment shows the model stays useful without age.

Chapter 5

CONCLUSION AND FUTURE WORK

5.1 Introduction

This final chapter brings the study together. It summarizes the main findings, explains the contributions, discusses the limitations, and suggests directions for future work. The purpose is to show not only what the model achieved, but also what the results mean for practical CVD screening.

5.2 Summary of the Study

I grabbed the Kaggle CVD dataset. Cleaned it up, threw out the impossible blood pressure readings, added some engineered stuff like BMI and hypertension flags. Then I trained seven different ML models – logistic regression, random forest, SVM, XGBoost, LightGBM, a stacking ensemble, and a simple neural net. Tuned every one of them with grid search and cross-validation. After all that XGBoost came out on top with 73.4% accuracy and an AUC of 0.801. Stacking gave a tiny AUC boost to 0.8023 but I stuck with XGBoost – it's simpler and faster. On top of the model I built a FastAPI service that spits out a probability and SHAP explanations so you can see why it made the call. The frontend is basic HTML, nothing fancy, but it works. Code's on GitHub, fully reproducible on any half-decent laptop.

5.3 Key Findings

So what did I actually learn? First off, machine learning can definitely spot CVD risk from everyday measurements like age, blood pressure, and cholesterol. That's not magic – it's just patterns. The gradient boosting methods (XGBoost, LightGBM) beat the older models by a few percent, and the difference is consistent across all metrics. Systolic blood pressure is the big boss – it always comes out as the most important feature, followed by cholesterol and age. That totally matches what doctors have been telling us for decades. Even if you take away age, the model still catches 66% of sick people, which is decent when data is missing. If I lower the threshold from 0.5 to 0.3 recall jumps to 89%. That means we catch almost 9 out of 10 sick patients even though we scare a few healthy ones. For a non-invasive screening tool that's a trade-off worth making. And the SHAP explanations – they show exactly what pushed the risk up or down, which makes the whole thing transparent and a lot easier to trust. Every prediction has a story behind it, not just a number.

5.4 Contributions of the Study

I think the main things I brought to the table are a really broad comparison. Most papers test two or three algorithms; I did seven on the same large dataset. That's a fair fight. I also checked what happens if you only have a few basic features – and it still works. That's important because real medical forms are often missing columns. Then there's the web API – it's not just a notebook that you run once; you can actually call it from a frontend and get a real prediction with SHAP. And everything is public. The code, the trained model, the whole pipeline. If someone wants to build on it, they just clone the repo and go. No cloud subscription needed – it runs on a regular laptop. That matters for small hospitals or labs that don't have big budgets. I also think the student-friendly style of this thesis shows that you don't need a PhD to try machine learning in medicine; a bit of Python and curiosity go a long way.

Several limitations should be considered. The study used one public dataset, so the results may not fully generalize to other populations, countries, or hospital systems. External validation was not performed, and the test split is not the same as testing on completely separate clinical records. The feature set is also limited to structured questionnaire and measurement variables; it does not include ECG signals, imaging, genetic markers, or detailed laboratory results.

The model should be treated as a screening aid, not as a replacement for clinical diagnosis. A 73% accuracy level is promising for a student-scale prototype, but it is not sufficient for autonomous medical decision-making. The web interface is also basic and would need user authentication, logging, monitoring, and clinical review before any real deployment. Finally, the interpretability work uses feature importance and SHAP explanations, but a deeper fairness and error analysis would be valuable before using the system in practice.

5.5 Future Work

If I had more time I'd test the model on Framingham and MIMIC-III just to see if it holds up. Maybe try some deep learning on tabular data – TabNet or NODE – they might squeeze out a few extra points. Adding ECG or imaging features would be a game-changer. I'd also deploy the API on a cloud server and run a pilot with a real clinic, get feedback from doctors, and see how it performs in the wild. Integrate LIME and SHAP more deeply into the UI so you can hover over a risk score and instantly see what's driving it. A mobile app would be nice – most patients have smartphones, why not let them check their own risk? And finally, I'd want to check fairness – split the performance by gender and age groups to make sure the model doesn't disadvantage anyone. Maybe even retrain with some fairness constraints if needed. The code is ready, it just needs someone to push it further.

5.6 Final Conclusion

This thesis showed that cardiovascular disease risk can be predicted from simple clinical and lifestyle measurements using machine learning. The best model is not perfect, and it should never replace a doctor, but it can support early screening by highlighting patients who may need further attention.

The most important lesson is that model building is only one part of a useful system. Data cleaning, feature engineering, careful evaluation, threshold selection, and interpretability all matter. By combining these steps with a working API, the project becomes more than an experiment: it becomes a reproducible prototype for practical decision support.

> **Takeaway:** This thesis showed that ML-based CVD prediction is feasible and can be deployed openly. Seven models were compared, XGBoost performed best, and blood pressure emerged as the top predictor. The code and API are public, ready for others to build on.

References

[1] World Health Organization (WHO). Cardiovascular diseases (CVDs) fact sheet. Accessed 2026. `https://www.who.int/`

[2] Kaggle. Cardiovascular Disease Dataset (`sulianova/cardiovascular-disease-dataset`). Accessed 2026. `https://www.kaggle.com/datasets/sulianova/cardiovascular-disease-dataset`

[3] P. K. Whelton, R. M. Carey, W. S. Aronow, et al. 2017 ACC/AHA/AAPA/ABC/ACPM/AGS/APhA/ASH/ASPC/NMA/PCNA guideline for the prevention, detection, evaluation, and management of high blood pressure in adults. *Hypertension*, 71(6):e13–e115, 2018.

[4] F. Visseren, F. J. M. Mach, Y. Smulders, et al. 2021 ESC Guidelines on cardiovascular disease prevention in clinical practice. *European Heart Journal*, 42(34):3227–3337, 2021.

[5] World Health Organization (WHO). Obesity: preventing and managing the global epidemic. WHO Technical Report Series 894. World Health Organization, 2000.

[6] T. Chen and C. Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2016.

[7] G. Ke, Q. Meng, T. Finley, et al. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[8] L. Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[9] C. Cortes and V. Vapnik. Support-vector networks. *Machine Learning*, 20:273–297, 1995.

[10] D. E. Rumelhart, G. E. Hinton, and R. J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986.

[11] S. M. Lundberg and S.-I. Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[12] F. Pedregosa, G. Varoquaux, A. Gramfort, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[13] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of KDD*, 2019.

[14] J. A. Hanley and B. J. McNeil. The meaning and use of the area under a receiver operating characteristic (ROC) curve. *Radiology*, 143(1):29–36, 1982.

[15] R. Kohavi. A study of cross-validation and bootstrap for accuracy estimation and model selection. In *Proceedings of IJCAI*, 1995.

[16] A. Niculescu-Mizil and R. Caruana. Predicting good probabilities with supervised learning. In *Proceedings of ICML*, 2005.

[17] S. Ramirez. FastAPI. Accessed 2026. `https://fastapi.tiangolo.com/`

[18] T. M. H. Uvicorn ASGI server. Accessed 2026. `https://www.uvicorn.org/`

[19] D. H. Wolpert. Stacked generalization. *Neural Networks*, 5(2):241–259, 1992.

[20] J. H. Friedman. Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5):1189–1232, 2001.

[21] J. Platt. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers*, 1999.

[22] The pandas development team. pandas: powerful Python data analysis toolkit. Accessed 2026. `https://pandas.pydata.org/`

[23] C. R. Harris, K. J. Millman, S. J. van der Walt, et al. Array programming with NumPy. *Nature*, 585:357–362, 2020.

[24] J. D. Hunter. Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.

[25] M. Waskom. seaborn: statistical data visualization. *Journal of Open Source Software*, 6(60):3021, 2021.

[26] Joblib development team. Joblib: running Python functions as pipeline jobs. Accessed 2026. `https://joblib.readthedocs.io/`

[27] S. M. Lundberg, G. Erion, H. Chen, et al. From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence*, 2:56–67, 2020.

[28] F. Doshi-Velez and B. Kim. Towards a rigorous science of interpretable machine learning. *arXiv:1702.08608*, 2017.

[29] W. Samek, G. Montavon, A. Vedaldi, L. K. Hansen, and K.-R. Müller. *Explainable AI: Interpreting, Explaining and Visualizing Deep Learning*. Springer, 2019.

[30] R. Detrano, A. Janosi, W. Steinbrunn, et al. International application of a new probability algorithm for the diagnosis of coronary artery disease. *The American Journal of Cardiology*, 64(5):304–310, 1989.

Appendix A

Implementation Details

A.1 System Architecture

Figure A.1 shows the pipeline.

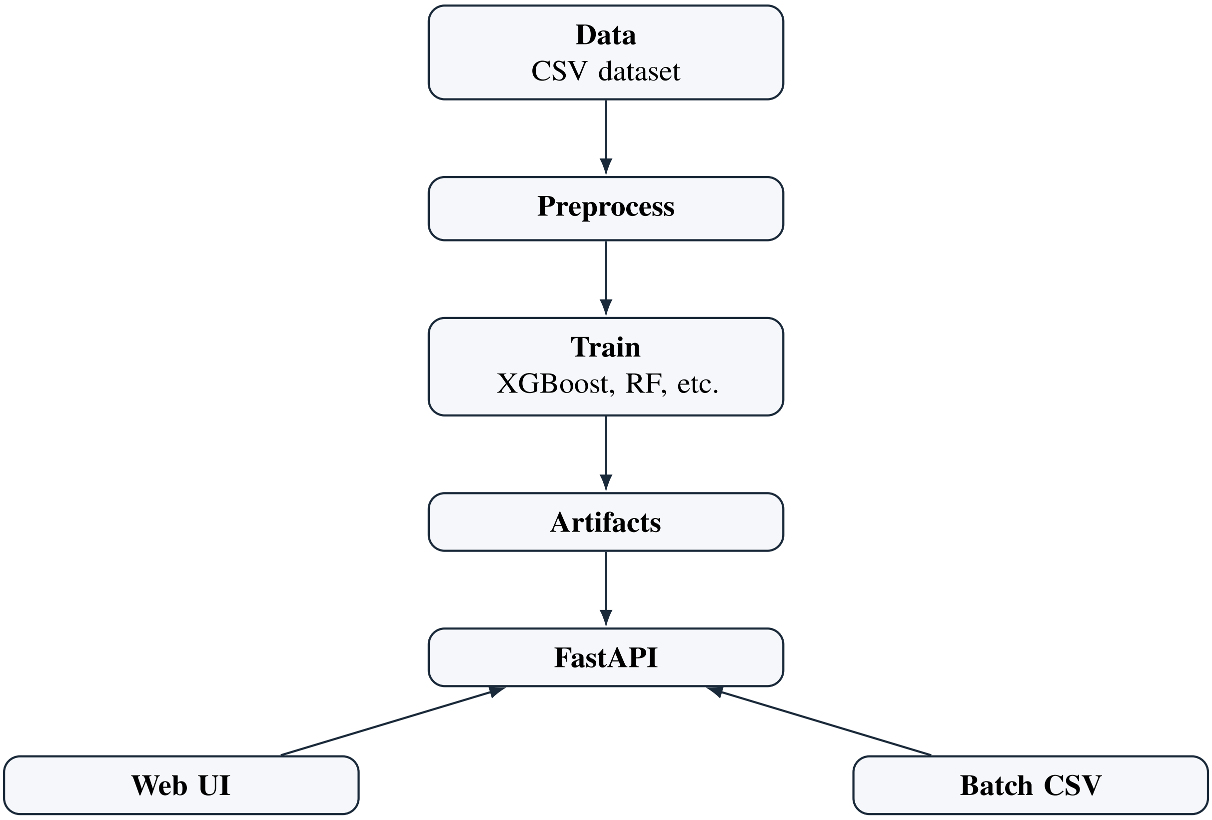

Figure A.1: System architecture.

A.2 Code Repository

The code is at `https://github.com/AlamSajjad456/R1`. Key files:

A.3 Training Commands

The command that actually built my model is this one.

```
python -m ml_system.train "data/cardio_train_clean.csv" --delimiter ";" --targe
```

And to fire up the API I use this.

```
python -m uvicorn ml_system.api:app --host 0.0.0.0 --port 8000
```

That's it. Two commands and the whole thing is running on my laptop.

A.4 Reproducibility Checklist

I made a checklist so I wouldn't forget steps. If you follow it you should get the same numbers I got.

1. Grab the original CVD CSV file from Kaggle and note the exact source.
2. Remove rows where blood pressure makes no sense before training anything.
3. Convert age from days to years before any analysis.
4. Build extra features only from the original columns don't pull in outside info.
5. Keep the test set completely separate until final evaluation.
6. Fix the random seed so splits and model training are identical every run.
7. Save the best model and the exact preprocessing steps after tuning.
8. Report accuracy precision recall F1 ROC-AUC and the confusion matrix.
9. Explain what threshold you picked and why not just stick with 0.5 blindly.

Honestly this checklist saved me a couple times. Without it I would've mixed up test data or forgotten to set the seed. Reproducibility matters a lot to me so I documented everything.

A.5 Model Card and Deployment Notes

I wrote down what the model is supposed to do. It takes age blood pressure cholesterol glucose smoking alcohol and physical activity. It gives back a risk score and a prediction. It does not give a diagnosis. It's a screening tool nothing more.

The people I imagine using it are other students researchers or devs who want to test a screening workflow. It is not for doctors making life-or-death decisions. If someone wants to use this in a real clinic they would need external validation proper privacy checks user login logs and monitoring. I didn't do any of that yet.

A.6 Limitations and Safety

Again this is academic only. I am not a medical professional and this is not a medical device. Don't use it on real patients without proper testing.